



JavaScript Fundamentals

Assoc. Prof. Dr. Kanda Runapongsa Saikaew
(krunapon@kku.ac.th)
Dept. of Computer Engineering
Khon Kaen University



Agenda

- Running JavaScript: browser and Node
- Code structure
- Variables and data types
- Type Conversions
- Basic operators, maths
- Loops: while and for
- The “switch” statement
- Functions



What is Node? (1/2)

- Node.js is a program that runs JavaScript on your computer, outside the browser.
 - JavaScript used to run **only inside a browser** (to make web pages interactive)
 - Node.js takes the **V8 engine** — the part of Chrome that executes JavaScript — and lets it run on its own, from the **terminal**.
 - So Node is a **runtime**: an environment that runs your JS code
- **Outside the browser, two things change:**
 - **Loses** browser-only features: no **window**, no **document**, no DOM, no **alert** / **prompt** / **confirm** — there's no web page to act on
 - **Gains** computer-level abilities the browser blocks for safety: read/write files, open network connections, run a **web server**.



What is Node? (2/2)

- **Why it matters here:**
 - Browser → **frontend** (what the user sees)
 - Node → **backend** (server, API, database) — we build this later
 - A full-stack app uses **both**.
- **Common misconceptions to clear up:**
 - Node is **not** a programming language — the language is JavaScript.
 - Node is **not** a framework — Express (later) is a framework that runs *on top of* Node.



Where JavaScript runs: Browser vs. Node

Same language, two runtimes:

- **Browser** (Chrome, Firefox...): has the DOM — **window**, **document**, and the UI popups **alert**, **prompt**, **confirm**. Used to make web pages interactive.
- **Node.js**: runs JavaScript outside the browser, in the terminal. No DOM, no **window**, and **no alert / prompt / confirm**. Used for tools, servers, and APIs (later in this course).
- The core language — variables, operators, loops, functions — works in both.
- Rule of thumb: **DOM / UI code** → **browser**; **pure logic** → **either**.



Input and Output in Browser and Node.js

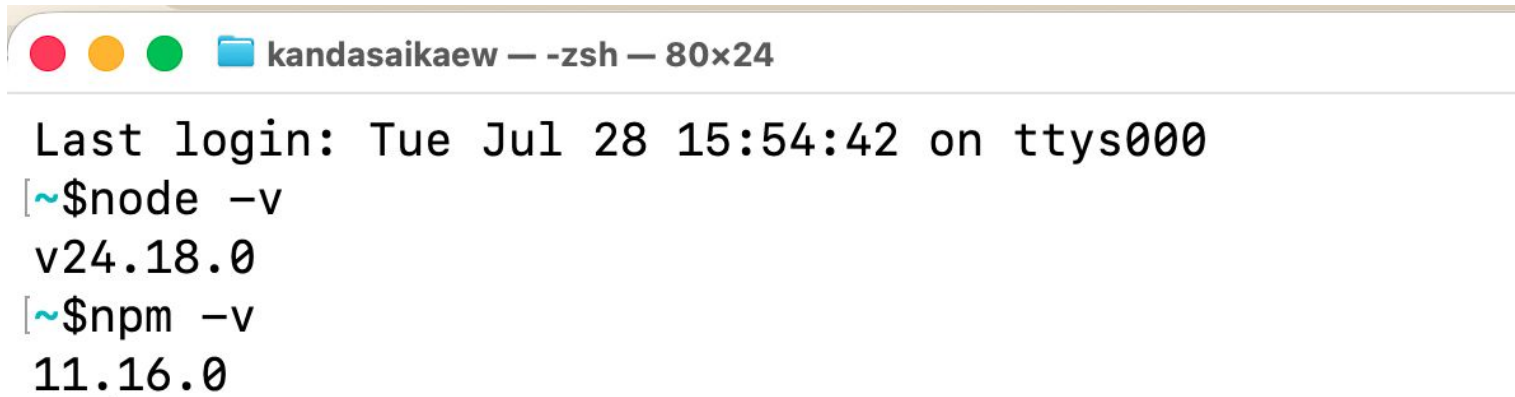
	Browser	Node.js
Output	<code>alert()</code> , <code>document</code>	<code>console.log()</code>
Input	<code>prompt()</code>	<code>readline</code> (built-in)
DOM?	Yes	No
Runs in	a web page	the terminal

- `prompt("Age?")` works in the browser but throws `ReferenceError: prompt is not defined` in Node



How to install Node

- Go to <https://nodejs.org> and download the **LTS** version (currently Node.js 24)
 - *LTS = Long-Term Support = stable.*
- Run the installer — accept the defaults, keep **"Add to PATH"** checked.
- Open a terminal (in VS Code: **Terminal** → **New Terminal**) and verify:

A screenshot of a terminal window with a title bar showing three colored circles (red, yellow, green) and a folder icon, followed by the text "kandasaikaew — -zsh — 80x24". The terminal content shows the last login time and the output of two commands: "node -v" and "npm -v".

```
kandasaikaew — -zsh — 80x24

Last login: Tue Jul 28 15:54:42 on ttys000
[~]$node -v
v24.18.0
[~]$npm -v
11.16.0
```



Install Code Runner Extension

- In VS code, install Code Runner Extension



Running JavaScript using Node



The screenshot shows the Visual Studio Code editor interface. At the top, a tab is labeled 'JS hello.js U X'. The editor area contains the following JavaScript code:

```
1 let name = "KKU";  
2 console.log("Helo " + name);  
3
```

Below the code editor, the 'OUTPUT' panel is active, displaying the execution results:

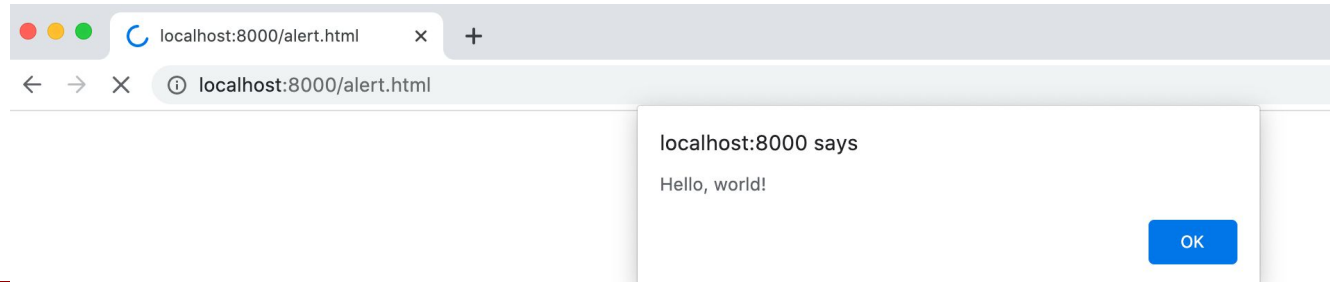
```
[Running] node "/Users/kandasaikaew/courses/en813701/webapp-demos/javascript/  
hello.js"  
Helo KKU  
  
[Done] exited with code=0 in 0.204 seconds
```



The "script" tag

- JavaScript programs can be inserted into any part of an HTML document with the help of the `<script>` tag

```
<!DOCTYPE HTML>
<html>
<body>
  <p>Before the script...</p>
  <script>
    alert("Hello, world!");
  </script>
  <p>...After the script</p>
</body>
</html>
```



External scripts

- If we have a lot of JavaScript code, we can put it into a separate file
- Script files are attached to the HTML with the src attribute

```
1 <script src="/path/to/script.js"></script>
```

- We can give a full URL as well

```
1 <script src="https://cdnjs.cloudflare.com/ajax/libs/lodash.js/4.17.11/lodash.js"></script>
```

- To attach several scripts, use multiple tags

```
1 <script src="/js/script1.js"></script>
2 <script src="/js/script2.js"></script>
3 ...
```



Internal vs. external scripts

- As a rule, only the simplest scripts are put into HTML
- More complex ones reside in separate files
- The benefit of a separate file is that the browser will download it and store it in its cache
- Other pages that reference the same script will take it from the cache instead of downloading it, so the file is actually downloaded only once
 - That reduces traffic and makes pages faster



If src is set, the script content is ignored

- A single <script> tag can't have both the src attribute and code inside

```
1 <script src="file.js">
2   alert(1); // the content is ignored, because src is set
3 </script>
```

- We must choose either an external <script src=""> or a regular <script> with code

```
1 <script src="file.js"></script>
2 <script>
3   alert(1);
4 </script>
```



Statements

```
1 alert("There will be an error")
2
3 [1, 2].forEach(alert)
```

```
1 alert("All fine now");
2
3 [1, 2].forEach(alert)
```

- The error in the no-semicolon variant occurs because JavaScript does not assume a semicolon before square brackets [...].
- Because the semicolon is not auto-inserted, the code in the first example is treated as a single statement.
- We recommend putting semicolons between statements even if they are separated by newlines.
- This rule is widely adopted by the community.



Comments

```
1 // This comment occupies a line of its own
2 alert('Hello');
3
4 alert('World'); // This comment follows the statement
```

```
1 /* An example with two messages.
2 This is a multiline comment.
3 */
4 alert('Hello');
5 alert('World');
```

Nested comments are not supported!

```
1 /*
2   /* nested comment ?!? */
3 */
4 alert( 'World' );
```



Variables

- A variable is a “named storage” for data
- We can use variables to store goodies, visitors, and other data
- To create a variable in JavaScript, use the let keyword
- The statement below creates (in other words: declares a variable with the name “message”)

```
1 let message;
```



Using let

```
1 let message = 'Hello!'; // define the variable and assign the value
2
3 alert(message); // Hello!
```

For the sake of better readability, please use a single line per variable. The multiline variant is a bit longer, but easier to read

```
1 let user = 'John';
2 let age = 25;
3 let message = 'Hello';
```

Some people also define multiple variables in this multiline style

```
1 let user = 'John',
2     age = 25,
3     message = 'Hello';
```



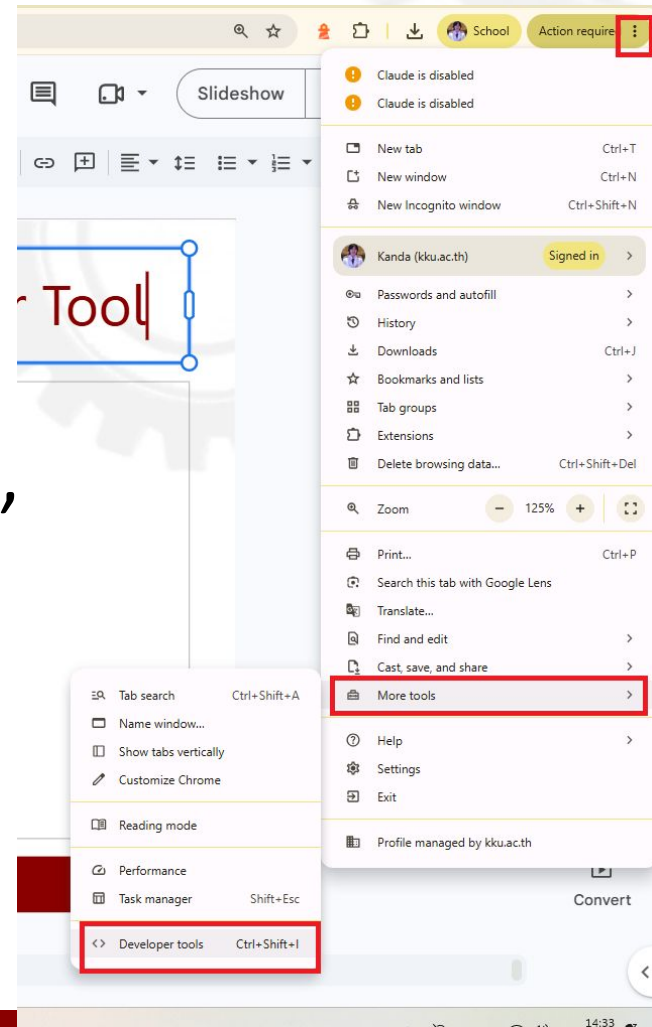
var instead of let

- In older scripts, you may also find another keyword: var instead of let
- The differences between var and let
 - var variables have no block scope, they are visible minimum at the function level
 - var declarations are processed at function start (script start for globals)
 - var variables can be declared below their use



Using Chrome Developer Tool

- Click three dots on the right top Chrome web browser
- Click at “More tools”
- Click at “Developer tools”



var vs. let

- Click at tab “console”

The image shows a development environment with VS Code on the left and Chrome DevTools on the right. In VS Code, the file `letvar.html` is open, showing an HTML document with a script block. The script contains the following code:

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4   <script>
5     console.log("1 " + x);
6     var x = 5;
7     console.log("2 " + x);
8
9     console.log("3 " + y);
10    let y = 5;
11    console.log("4 " + y);
12  </script>
13 </body>
```

In Chrome DevTools, the 'Console' tab is selected, showing the following error:

```
1 undefined letvar.html:5
2 5 letvar.html:7
Uncaught ReferenceError: Cannot access 'y' before initialization
    at letvar.html:9:28
Live reload enabled. letvar.html:41
content script loaded content.903826e2.js:1
Failed to load resource: the server responded with a status of 404 (Not Found) favicon.ico:1
```

The error message indicates a 'ReferenceError: Cannot access 'y' before initialization' at line 9, column 28 of `letvar.html`. This is because the variable `y` is used before it is declared with `let` on line 10. The 'Console' tab in DevTools is highlighted with a red box.

var vs. let

- When using let, we need to declare a variable before accessing it

The image shows a development environment with VS Code on the left and Chrome DevTools on the right. In VS Code, the file `letvar.html` contains the following JavaScript code:

```
1 <html>
2 <body>
3   <script>
4     console.log(1 + x);
5     // console.log(2 + x);
6     var x = 5;
7     console.log("2 " + x);
8     // console.log("3 " + y);
9     let y = 5;
10    console.log("4 " + y);
11  </script>
12 </body>
13 </html>
```

The Chrome DevTools console shows the following error messages:

```
1 undefined letvar.html:5
2 5 letvar.html:7
4 5 letvar.html:11
content script loaded content.983826e2.js:1
```

The error occurs because the `let` variable `y` is not yet declared when the first `console.log` statement is executed, illustrating the concept of variable hoisting.



Variable naming

- There are two limitations on variable names in JavaScript
 - The name must contain only letters, digits, or the symbols \$ and _
 - The first character must not be a digit
- When the name contains multiple words, camelCase is commonly used
- That is: words go one after another, each word except first starting with a capital letter:
myVeryLongName

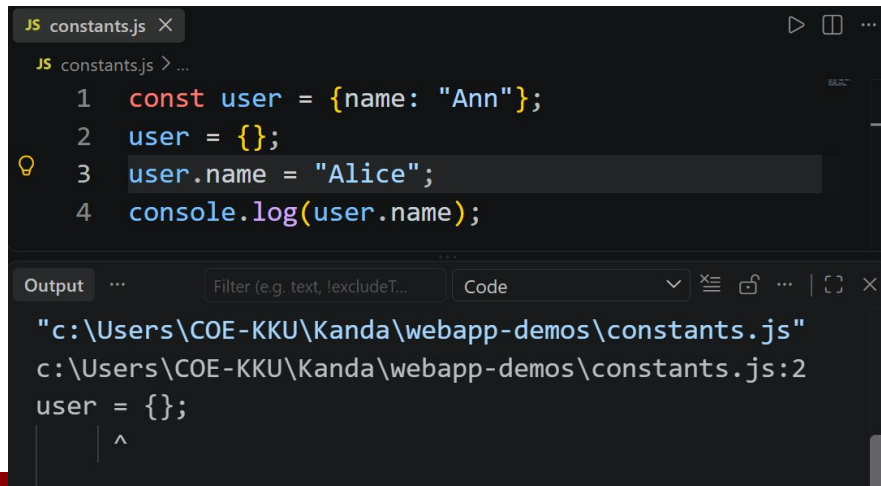


Constants

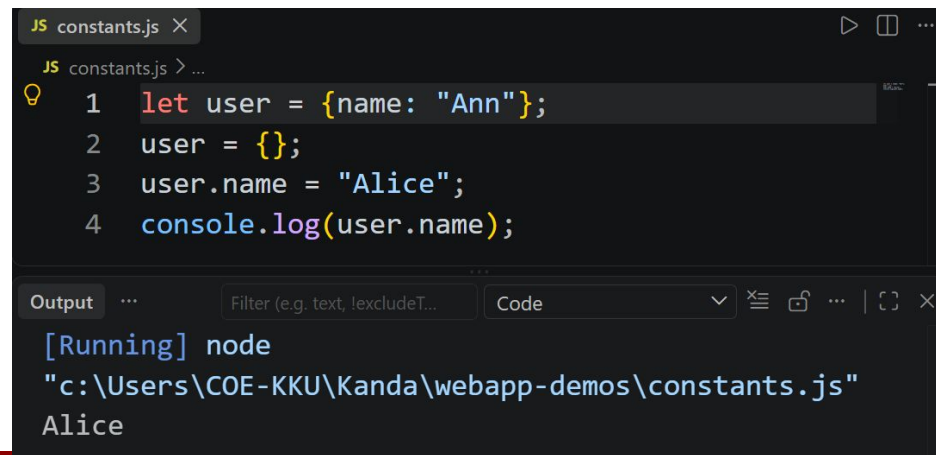
- To declare a constant (unchanging) variable, use `const` instead of `let`

```
1 const myBirthday = '18.04.1982';  
2  
3 myBirthday = '01.01.2001'; // error, can't reassign the constant!
```

- `const` prevents **re-assigning** the variable, but the contents of an object/array it points to can still change



```
JS constants.js X  
JS constants.js > ...  
1 const user = {name: "Ann"};  
2 user = {};  
3 user.name = "Alice";  
4 console.log(user.name);  
  
Output ... Filter (e.g. text, !excludeT... Code  
"c:\Users\COE-KKU\Kanda\webapp-demos\constants.js"  
c:\Users\COE-KKU\Kanda\webapp-demos\constants.js:2  
user = {};  
  ^
```



```
JS constants.js X  
JS constants.js > ...  
1 let user = {name: "Ann"};  
2 user = {};  
3 user.name = "Alice";  
4 console.log(user.name);  
  
Output ... Filter (e.g. text, !excludeT... Code  
[Running] node  
"c:\Users\COE-KKU\Kanda\webapp-demos\constants.js"  
Alice
```



Uppercase constants

- There is a widespread practice to use constants as aliases for difficult-to-remember values that are known prior to execution.
- Such constants are named using capital letters and underscores

```
1  const COLOR_RED = "#F00";
2  const COLOR_GREEN = "#0F0";
3  const COLOR_BLUE = "#00F";
4  const COLOR_ORANGE = "#FF7F00";
5
6  // ...when we need to pick a color
7  let color = COLOR_ORANGE;
8  alert(color); // #FF7F00
```

- In other words, capital-named constants are only used as aliases for “hard-coded” values.



Name things right

- Use human-readable names like `userName` or `shoppingCart`
- Stay away from abbreviations or short names like `a`, `b`, `c`
- Make names maximally descriptive and concise
 - Examples of bad names are `data` and `value`
- Agree on terms within our team and in your own mind
 - If a site visitor is called a `user`, we may want to use variables `currentUser` or `newUser` instead of `currentVisitor` or `newManInTown`



Data types

- A value in JavaScript is always of a certain type
- There are eight basic data types in JavaScript
- We can put any type in a variable
- For example, a variable can at one moment be a string and then store a number

```
1 // no error
2 let message = "hello";
3 message = 123456;
```



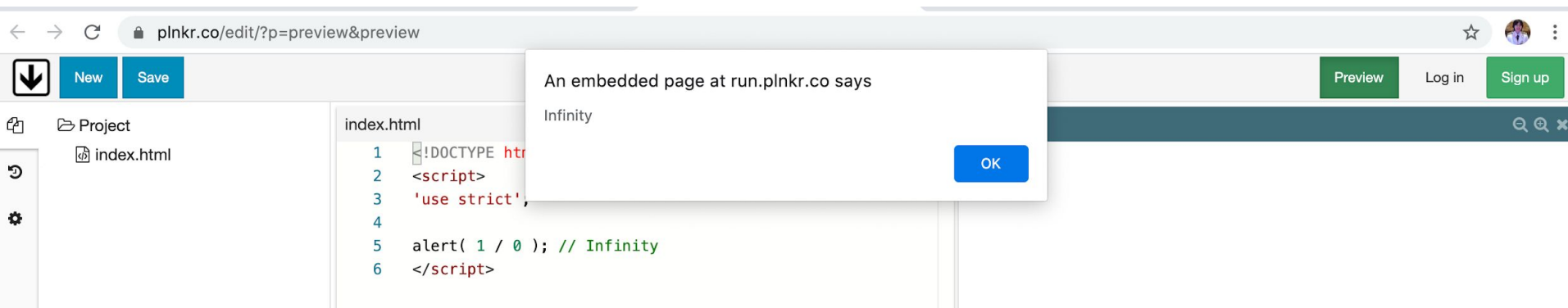
8 basic data types in JavaScript

- Number for numbers of any kind:
- BigInt for integer numbers of arbitrary length
- String
- Boolean (logical type) for true/false
- null for unknown values
- undefined for unassigned values
- Object for more complex data structures
- Symbol for unique identifiers



Number

- The *number* type represents both integer and floating point numbers
- Besides regular numbers, there are so-called “special numeric values” which also belong to this data type: Infinity, -Infinity and NaN
- <https://plnkr.co/edit/?p=preview&preview>



BigInt

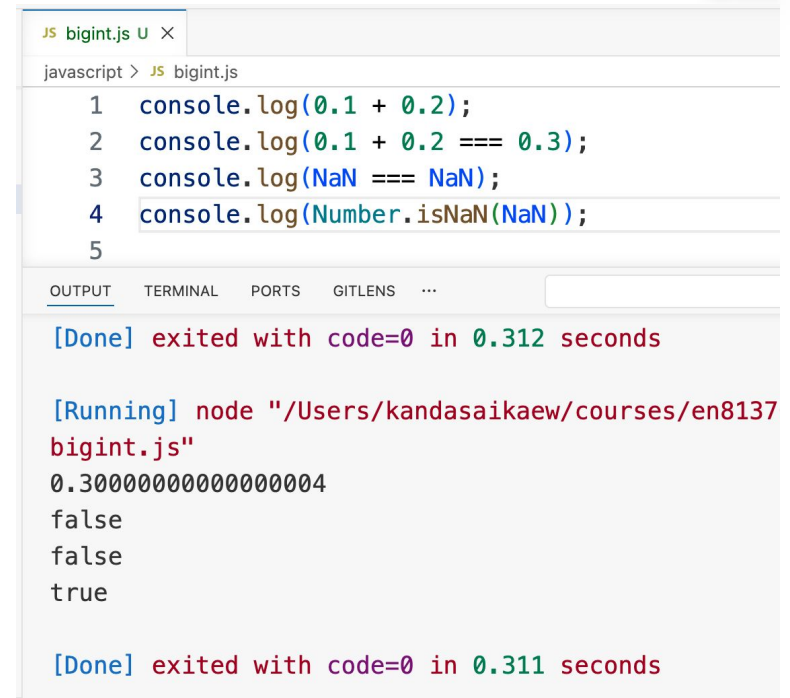
- In JavaScript, the “number” type cannot represent integer values larger than $(2^{53} - 1)$ or less than $-(2^{53} - 1)$ for negatives

```
1 // the "n" at the end means it's a BigInt
2 const bigInt = 1234567890123456789012345678901234567890n;
```



Number: precision & NaN

- Numbers are stored in binary, so some decimals are imprecise
- NaN** (“Not a Number”) propagates: any math with **NaN** gives **NaN**
- NaN** is the only value not equal to itself - test with **Number.isNaN()**



```
JS bigint.js U X
javascript > JS bigint.js
1 console.log(0.1 + 0.2);
2 console.log(0.1 + 0.2 === 0.3);
3 console.log(NaN === NaN);
4 console.log(Number.isNaN(NaN));
5

OUTPUT TERMINAL PORTS GITLENS ...
[Done] exited with code=0 in 0.312 seconds

[Running] node "/Users/kandasaikaew/courses/en8137/bigint.js"
0.30000000000000004
false
false
true

[Done] exited with code=0 in 0.311 seconds
```



String

- A string in JavaScript must be surrounded by quotes

```
1 let str = "Hello";  
2 let str2 = 'Single quotes are ok too';  
3 let phrase = `can embed another ${str}`;
```

- Double and single quotes are “simple” quotes. There’s practically no difference between them in JavaScript.
- Backticks are “extended functionality” quotes. They allow us to embed variables and expressions into a string by wrapping them in `${...}`

```
1 let name = "John";  
2  
3 // embed a variable  
4 alert( `Hello, ${name}!` ); // Hello, John!  
5  
6 // embed an expression  
7 alert( `the result is ${1 + 2}` ); // the result is 3
```



Boolean (logical type)

- The boolean type has only two values: true and false

```
1 let nameFieldChecked = true; // yes, name field is checked
2 let ageFieldChecked = false; // no, age field is not checked
```

```
1 let isGreater = 4 > 1;
2
3 alert( isGreater ); // true (the comparison result is "yes")
```



The “null” value

```
1 let age = null;
```

- In JavaScript, null is not a “reference to a non-existing object” or a “null pointer” like in some other languages.
- It’s just a special value which represents “nothing”, “empty” or “value unknown”.
- The code above states that age is unknown.



The “undefined” value

- The meaning of undefined is “value is not assigned”
- If a variable is declared, but not assigned, then its value is undefined

```
1  let age;  
2  
3  alert(age); // shows "undefined"
```



Objects and symbols

- The object type is special
- All other types are called “primitive” because their values can contain only a single thing
- In contrast, objects are used to store collections of data and more complex entities
- The symbol type is used to create unique identifiers for objects



The typeof operator

- The typeof operator returns the type of the argument
- It supports two forms of syntax
 - As an operator: typeof x
 - As a function: typeof(x)

```
1  typeof undefined // "undefined"
2
3  typeof 0 // "number"
4
5  typeof 10n // "bigint"
6
7  typeof true // "boolean"
8
9  typeof "foo" // "string"
10
11 typeof Symbol("id") // "symbol"
12
13 typeof Math // "object" (1)
14
15 typeof null // "object" (2)
16
17 typeof alert // "function" (3)
```



Interaction: alert, prompt, confirm

- alert
 - Shows a message
- prompt
 - Shows a message asking the user to input text
 - It returns the text or, if Cancel button or Esc is clicked, null
- confirm
 - Shows a message and waits for the user to press “OK” or “Cancel”
 - It returns true for OK and false for Cancel/Esc



Sample code of prompt and alert

```
prompt-box.html X
prompt-box.html > html > head > script > text
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <script>
5       let text = prompt("Please enter text");
6       document.write("You have entered " + text);
7     </script>
8   </head>
9 </html>
```

127.0.0.1:5500 says

Please enter text

OK Cancel

← → ↻ ⓘ 127.0.0.1:5500/prompt-box.html

You have entered Manee



Sample code for confirm

<> confirmBox.html X

javascript > <> confirmBox.html > html > head

```
1 <html>
2   <head>
3     <script type="text/javascript">
4       confirm("Please click OK or Cancel");
5     </script>
6   </head>
7   <body></body>
8 </html>
```

< > X ⓘ 127.0.0.1:5500/javascript/confirmBox.html

127.0.0.1:5500 says

Please click OK or Cancel

OK

Cancel



Type Conversions: Numeric conversion

- Numeric conversion happens in mathematical functions and expressions automatically

```
1 alert( "6" / "2" ); // 3, strings are converted to numbers
```

```
1 let str = "123";  
2 alert(typeof str); // string  
3  
4 let num = Number(str); // becomes a number 123  
5  
6 alert(typeof num); // number
```

- If the string is not a valid number, the result of such a conversion is NaN

```
1 let age = Number("an arbitrary string instead of a number");  
2  
3 alert(age); // NaN, conversion failed
```



Type Conversions: String conversion

- String conversion happens when we need the string form of a value
 - `alert(value)` does it to show the value
 - `String(value)` function is to convert a value to a string

```
1 let value = true;
2 alert(typeof value); // boolean
3
4 value = String(value); // now value is a string "true"
5 alert(typeof value); // string
```



Type Conversions: Boolean conversion

- The conversion rule:
 - Values that are intuitively “empty”, like 0, an empty string, null, undefined, and NaN, become false.
 - Other values become true.

```
1 alert( Boolean(1) ); // true
2 alert( Boolean(0) ); // false
3
4 alert( Boolean("hello") ); // true
5 alert( Boolean("") ); // false
```

```
1 alert( Boolean("0") ); // true
2 alert( Boolean(" ") ); // spaces, also true (any non-empty string is true)
```



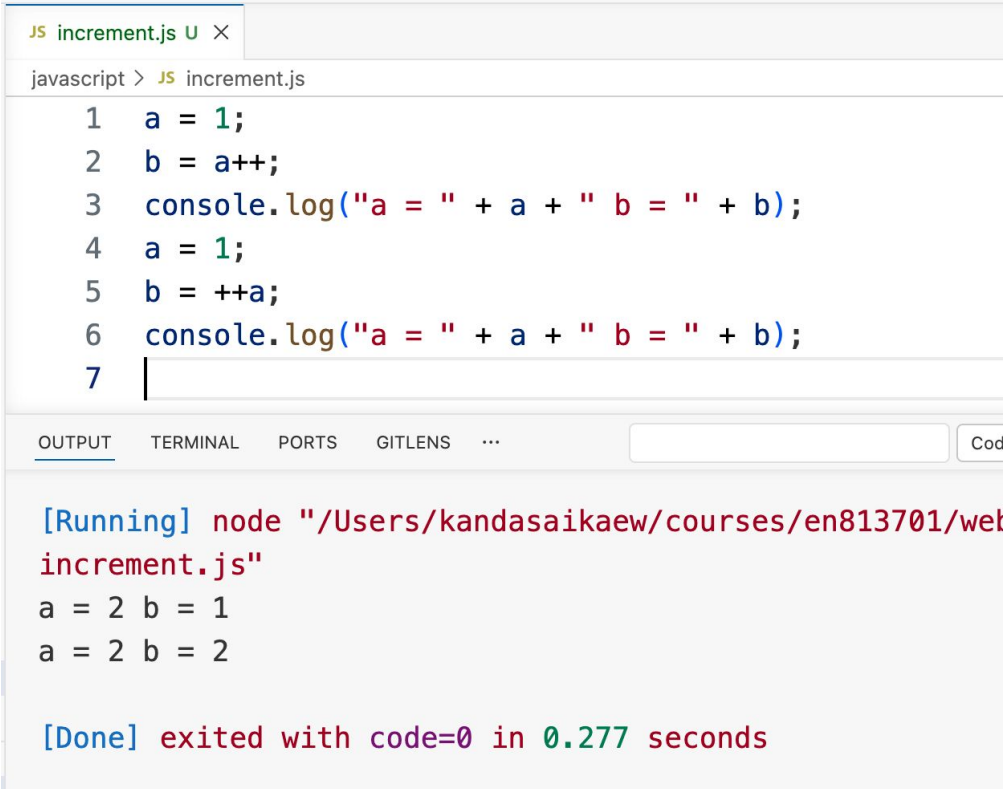
Operator Precedence

Precedence	Name	Sign
...	...	...
17	unary plus	+
17	unary negation	-
16	exponentiation	**
15	multiplication	*
15	division	/
13	addition	+
13	subtraction	-
...	...	...
3	assignment	=
...	...	...



Increment/Decrement

- `a++` returns the old value then increments
 - `b = a++`
 - `b = a; then a = a + 1;`
- `++a` increments then returns the new value
 - `b = ++a`
 - `a = a + 1; then b = a;`



The screenshot shows a code editor with a file named `increment.js`. The code contains two blocks of JavaScript code. The first block initializes `a` to 1 and `b` to `a++`, then logs the values. The second block resets `a` to 1, increments it to 2 with `++a`, and logs the values again. The output pane shows the execution results: `a = 2 b = 1` for the first log and `a = 2 b = 2` for the second log.

```
JS increment.js U X
javascript > JS increment.js

1 a = 1;
2 b = a++;
3 console.log("a = " + a + " b = " + b);
4 a = 1;
5 b = ++a;
6 console.log("a = " + a + " b = " + b);
7 |

OUTPUT TERMINAL PORTS GITLENS ... Cod

[Running] node "/Users/kandasaikaew/courses/en813701/web/increment.js"
a = 2 b = 1
a = 2 b = 2

[Done] exited with code=0 in 0.277 seconds
```



Increment/decrement among other operators

```
1 let counter = 1;  
2 alert( 2 * ++counter ); // 4
```

```
1 let counter = 1;  
2 alert( 2 * counter++ ); // 2, because counter++ returns the "old" value
```

We advise a style of “one line – one action”:

```
1 let counter = 1;  
2 alert( 2 * counter );  
3 counter++;
```



String comparison

- To see whether a string is greater than another, JavaScript uses the so-called “dictionary” or “lexicographical” order
- In other words, strings are compared letter by letter

```
1 alert( 'Z' > 'A' ); // true
2 alert( 'Glow' > 'Glee' ); // true
3 alert( 'Bee' > 'Be' ); // true
```



Strict equality

- A regular equality check `==` has a problem because it cannot differentiate 0 from false and cannot differentiate an empty string from false

```
1 alert( 0 == false ); // true
```

```
1 alert( '' == false ); // true
```

- A strict equality operator `===` checks the equality without type conversion.

```
1 alert( 0 === false ); // false, because the types are different
```



Comparison with null and undefined (1/2)

- There's a non-intuitive behavior when null or undefined are compared to other values
- For a strict equality check ===, these values are different because each of them is a different type

```
1 alert( null == undefined ); // true
```

```
1 alert( null === undefined ); // false
```



Comparison with null and undefined (2/2)

- `==` converts types before comparing
- `===` does not convert types -- it compares both value and type

```
JS comparison_null_undefined.js U X
javascript > JS comparison_null_undefined.js
1 console.log(0 == "");
2 console.log(0 == false);
3 console.log("" == false);
4 console.log(null == undefined);
5 console.log(null === undefined);
6 console.log(NaN == NaN);
7 |

OUTPUT TERMINAL PORTS GITLENS ...

[Running] node "/Users/kandasaikaew/courses
comparison_null_undefined.js"
true
true
true
true
false
false
```



The if statement

- The if(...) statement evaluates a condition in parentheses and, if the result is true, executes a block of code
- We recommend wrapping your code block with curly braces {} every time you use an if statement, even if there is only one statement to execute
 - Doing so improves readability.



Boolean conversion

- The if (...) statement evaluates the expression in its parentheses and converts the result to a boolean
- A number 0, an empty string "", null, undefined, and NaN all become false. Because of that they are called “falsy” values
- Other values become true, so they are called “truthy”



The else clause

- The if statement may contain an optional “else” block
- It executes when the condition is false

```
5   let year = prompt('In which year was the ECMAScript-2015
    specification published?', '');
6
7   if (year == 2015) {
8   |   alert( 'You guessed it right!' );
9   } else {
10  |   alert( 'How can you be so wrong?' ); // any value except
    |   2015
11  }
```



Several conditions: else if

- Sometimes, we'd like to test several variants of a condition. The else if clause lets us do that

```
5   let year = prompt('In which year was the ECMAScript-2015
6       specification published?', '');
7   if (year < 2015) {
8       alert( 'Too early...' );
9   } else if (year > 2015) {
10      alert( 'Too late' );
11  } else {
12      alert( 'Exactly!' );
13  }
```



Conditional operator '?'

- The so-called “conditional” or “question mark” is represented by a question mark ?
- Sometimes it’s called “ternary”, because the operator has three operands
- It is actually the one and only operator in JavaScript which has that many.

```
1 let result = condition ? value1 : value2;
```



The if statement vs conditional operator ?

- Using the if statement

```
4 if (age > 18) {  
5   accessAllowed = true;  
6 } else {  
7   accessAllowed = false;  
8 }
```

- We can omit the parentheses around `age > 18`. The question mark operator has a low precedence, so it executes after the comparison `>`

```
1 // the comparison operator "age > 18" executes first anyway  
2 // (no need to wrap it into parentheses)  
3 let accessAllowed = age > 18 ? true : false;
```

- You can avoid using the question mark operator because the comparison itself returns true/false

```
1 // the same  
2 let accessAllowed = age > 18;
```



Multiple '?'

- A sequence of question mark operators ? can return a value that depends on more than one condition

```
1 let age = prompt('age?', 18);
2
3 let message = (age < 3) ? 'Hi, baby!' :
4   (age < 18) ? 'Hello!' :
5   (age < 100) ? 'Greetings!' :
6   'What an unusual age!';
7
8 alert( message );
```



Logical operators

- There are three logical operators in JavaScript: || (OR), && (AND), ! (NOT)

```
1 result = value1 || value2 || value3;
```

- The OR || operator does the following:
- Evaluates operands from left to right.
- For each operand, converts it to boolean. If the result is true, stops and returns the original value of that operand.
- If all operands have been evaluated (i.e. all were false), returns the last operand.



OR "||" finds the first truthy value

- A value is returned in its original form, without the conversion.
- In other words, a chain of OR "||" returns the first truthy value or the last one if no truthy value is found.

```
1 alert( 1 || 0 ); // 1 (1 is truthy)
2
3 alert( null || 1 ); // 1 (1 is the first truthy value)
4 alert( null || 0 || 1 ); // 1 (the first truthy value)
5
6 alert( undefined || null || 0 ); // 0 (all falsy, returns the last value)
```

```
1 true || alert("not printed");
2 false || alert("printed");
```



AND "&&" finds the first falsy value

```
1 result = value1 && value2 && value3;
```

The AND && operator does the following:

- Evaluates operands from left to right.
- For each operand, converts it to a Boolean
 - If the result is false, stops and returns the original value of that operand.
- If all operands have been evaluated (i.e. all were truthy), returns the last operand.
- In other words, AND returns the first falsy value or the last value if none were found.



Example of using &&

```
1 // if the first operand is truthy,  
2 // AND returns the second operand:  
3 alert( 1 && 0 ); // 0  
4 alert( 1 && 5 ); // 5  
5  
6 // if the first operand is falsy,  
7 // AND returns it. The second operand is ignored  
8 alert( null && 5 ); // null  
9 alert( 0 && "no matter what" ); // 0
```



Precedence of AND && is higher than OR ||

- The precedence of AND && operator is higher than OR ||.
- So the code `a && b || c && d` is essentially the same as if the && expressions were in parentheses: `(a && b) || (c && d)`.



Nullish coalescing operator '??'

- The nullish coalescing operator ?? provides a short syntax for selecting a first “defined” variable from the list.
- The result of a ?? b is:
 - a if it’s not null or undefined,
 - b, otherwise.
- x = a ?? b is a short equivalent to

```
1 x = (a !== null && a !== undefined) ? a : b;
```



Example of using Nullish coalescing operator '??'

- Imagine, we have a user, and there are variables firstName, lastName or nickName for their first name, last name and the nick name
 - All of them may be undefined, if the user decided not to enter any value

```
1 let firstName = null;
2 let lastName = null;
3 let nickName = "Supercoder";
4
5 // show the first not-null/undefined value
6 alert(firstName ?? lastName ?? nickName ?? "Anonymous"); // Supercoder
```



Comparison ?? with ||

- The OR || operator can be used in the same way as ??
- Actually, we can replace ?? with || in the code above and get the same result
- The important difference is that:
 - || returns the first *truthy* value.
 - ?? returns the first *defined* value

```
1 let height = 0;
2
3 alert(height || 100); // 100
4 alert(height ?? 100); // 0
```

```
1 let x = (1 && 2) ?? 3; // Works
2
3 alert(x); // 2
```



Loops: while

- We often need to repeat actions
- Loops are a way to repeat the same code multiple times

```
1  let i = 0;
2  while (i < 3) { // shows 0, then 1, then 2
3      alert( i );
4      i++;
5  }
```



Loops: do ... while

- The loop will first execute the body, then check the condition, and, while it's truthy, execute it again and again

```
1  let i = 0;  
2  do {  
3    alert( i );  
4    i++;  
5  } while (i < 3);
```



Loops: for

```
1  for (begin; condition; step) {  
2    // ... loop body ...  
3  }
```

```
1  for (let i = 0; i < 3; i++) { // shows 0, then 1, then 2  
2    alert(i);  
3  }
```

part

begin	<code>i = 0</code>	Executes once upon entering the loop.
condition	<code>i < 3</code>	Checked before every loop iteration. If false, the loop stops.
body	<code>alert(i)</code>	Runs again and again while the condition is truthy.
step	<code>i++</code>	Executes after the body on each iteration.



Inline variable declaration

```
1  for (let i = 0; i < 3; i++) {  
2    alert(i); // 0, 1, 2  
3  }  
4  alert(i); // error, no such variable
```

```
1  let i = 0;  
2  
3  for (i = 0; i < 3; i++) { // use an existing variable  
4    alert(i); // 0, 1, 2  
5  }  
6  
7  alert(i); // 3, visible, because declared outside of the loop
```



Skipping parts

```
1 let i = 0; // we have i already declared and assigned
2
3 for (; i < 3; i++) { // no need for "begin"
4     alert( i ); // 0, 1, 2
5 }
```

```
1 let i = 0;
2
3 for (; i < 3;) {
4     alert( i++ );
5 }
```

```
1 for (;;) {
2     // repeats without limits
3 }
```



Breaking the loop

```
1  let sum = 0;
2
3  while (true) {
4
5      let value = +prompt("Enter a number", '');
6
7      if (!value) break; // (*)
8
9      sum += value;
10
11 }
12 alert( 'Sum: ' + sum );
```



Continue to the next iteration

- The continue directive is a “lighter version” of break
- It doesn't stop the whole loop
- Instead, it stops the current iteration and forces the loop to start a new one (if the condition allows)

```
1  for (let i = 0; i < 10; i++) {  
2  
3      // if true, skip the remaining part of the body  
4      if (i % 2 == 0) continue;  
5  
6      alert(i); // 1, then 3, 5, 7, 9  
7  }
```



Inner and Outer Loops

```
JS outer_inner.js X
JS outer_inner.js > j
1  for (let i = 0; i < 3; i++) { // outer loop
2      for (let j = 0; j < 2; j++) { // inner loop
3          console.log(`${i}, ${j}`);
4      }
5  }
```

Output ... Filter (e.g. text, !excludeText, t... Code

```
[Running] node
"c:\Users\COE-KKU\Kanda\webapp-demos\outer_inner.js"
(0, 0)
(0, 1)
(1, 0)
(1, 1)
(2, 0)
(2, 1)
```



Labels for break/continue

```
1  for (let i = 0; i < 3; i++) {  
2  
3    for (let j = 0; j < 3; j++) {  
4  
5      let input = prompt(`Value at coords (${i},${j})`, '');  
6  
7      // what if we want to exit from here to Done (below)?  
8    }  
9  }  
10  
11 alert('Done!');
```

- We need a way to stop the process if the user cancels the input.
- The ordinary break after input would only break the inner loop.
- That's not sufficient—labels, come to the rescue!



Label

- A *label* is an identifier with a colon before a loop
- The `break <labelName>` statement in the loop below breaks out to the label

```
1  outer: for (let i = 0; i < 3; i++) {  
2  
3    for (let j = 0; j < 3; j++) {  
4  
5      let input = prompt(`Value at coords (${i},${j})`, '');  
6  
7      // if an empty string or canceled, then break out of both loops  
8      if (!input) break outer; // (*)  
9  
10     // do something with the value...  
11   }  
12 }  
13 alert('Done!');
```



Labels do not allow to “jump” anywhere

- Labels do not allow us to jump into an arbitrary place in the code.
- For example, it is impossible to do this:

```
1 break label; // doesn't jumps to the label below
2
3 label: for (...)
```

- A call to break/continue is only possible from inside a loop and the label must be somewhere above the directive.



How to debug JavaScript (1/5)

- Sample code

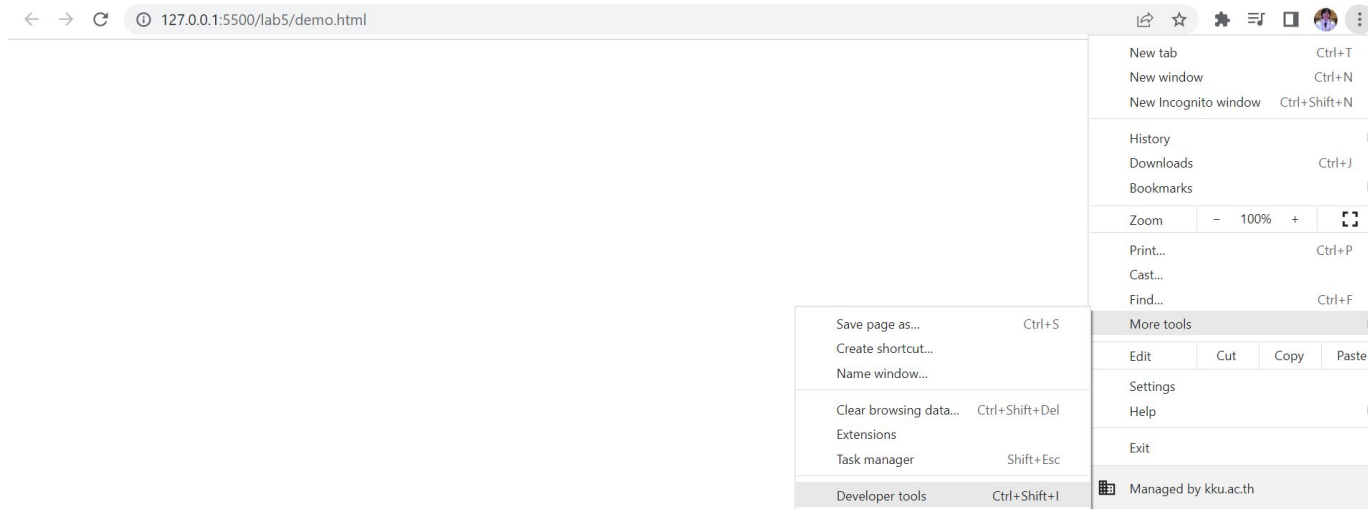
<> demo.html U X

lab5 > <> demo.html > head

```
1  <!DOCTYPE html>
2  <head>
3      <script>
4          let num1 = prompt("Enter the first number");
5          let num2 = prompt("Enter the second number");
6          sum = num1 + num2;
7          alert(`${num1} + ${num2} = {sum}`)
8      </script>
9  </head>
10 </html>
```

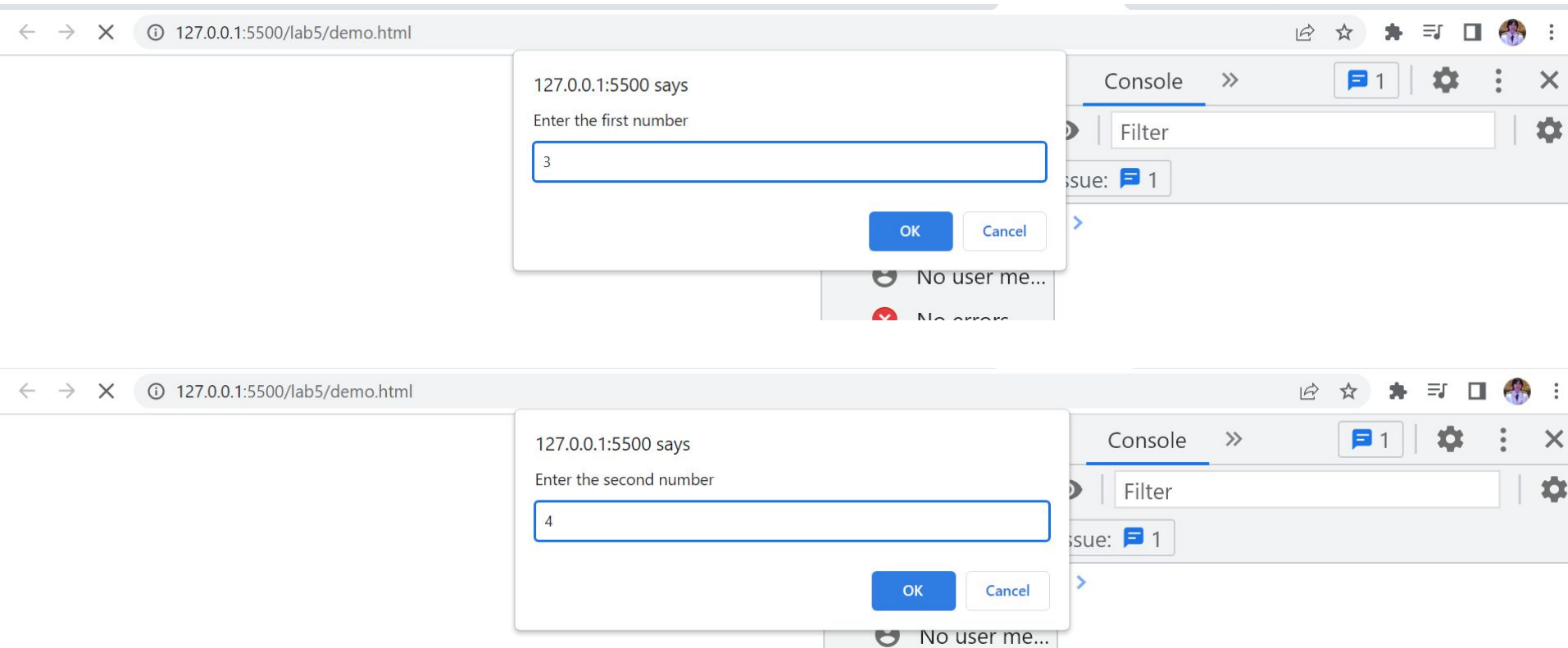
How to debug JavaScript (2/5)

- Run the web app on a server
- Then use Chrome developer tool to debug the program by press Ctrl-Shift+I or Go to menus as show below



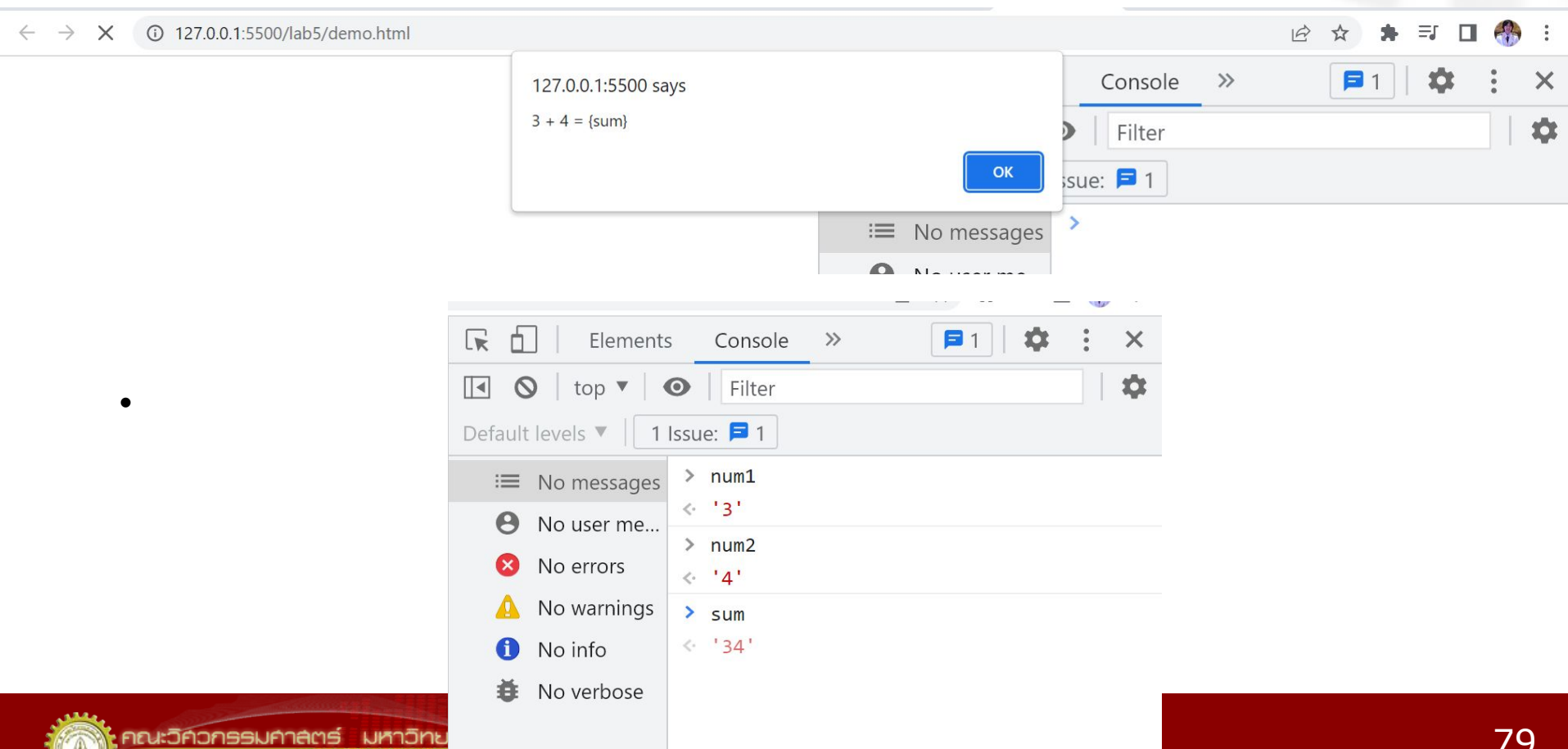
How to debug JavaScript (3/5)

- click at Console tab
- Enter the values prompted by the app

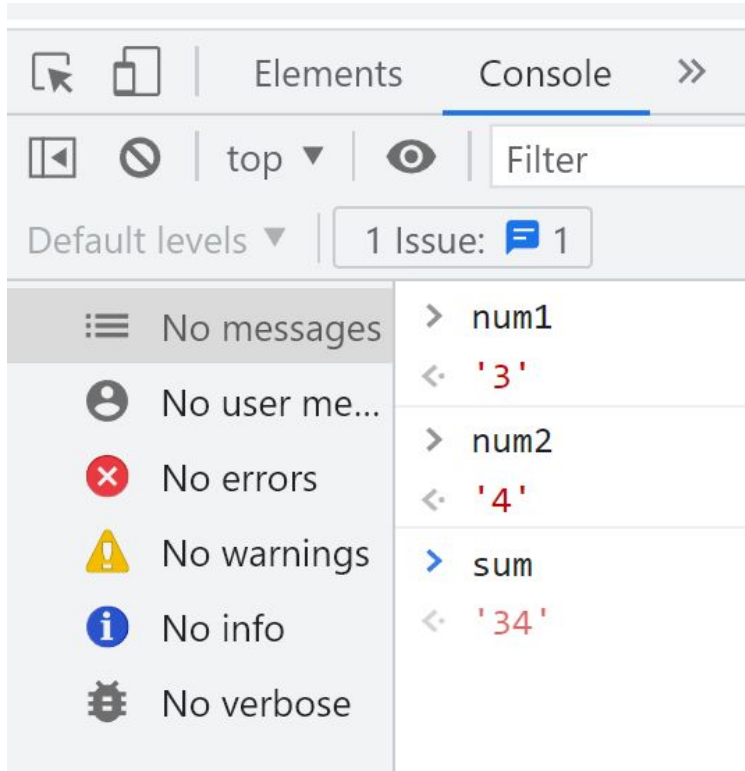


How to debug JavaScript (4/5)

- Run the web app and check the result



How to debug JavaScript (5/5)



- Bug1
 - num1 and num2 are strings
 - Thus, sum is the concatenation of num1 and num2
 - We need to convert the string to the number
- Bug2
 - The sum is not displayed on the web but we can see its value on the console
 - We need to look at the code whether we retrieve the value of the variable sum correctly



The switch statement

- A switch statement can replace multiple if checks.
- It gives a more descriptive way to compare a value with multiple variants

```
1  let a = 2 + 2;
2
3  switch (a) {
4    case 3:
5      alert( 'Too small' );
6      break;
7    case 4:
8      alert( 'Exactly!' );
9      break;
10   case 5:
11     alert( 'Too large' );
12     break;
13   default:
14     alert( "I don't know such values" );
15 }
```



If we don't use break

```
1  let a = 2 + 2;  
2  
3  switch (a) {  
4    case 3:  
5      alert( 'Too small' );  
6    case 4:  
7      alert( 'Exactly!' );  
8    case 5:  
9      alert( 'Too big' );  
10   default:  
11     alert( "I don't know such values" );  
12 }
```

```
1  alert( 'Exactly!' );  
2  alert( 'Too big' );  
3  alert( "I don't know such values" );
```



Grouping of "cases"

```
1  let a = 3;
2
3  switch (a) {
4      case 4:
5          alert('Right!');
6          break;
7
8      case 3: // (*) grouped two cases
9      case 5:
10         alert('Wrong!');
11         alert("Why don't you take a math class?");
12         break;
13
14     default:
15         alert('The result is strange. Really.');
```

```
16 }
```



Type matters

```
1 let arg = prompt("Enter a value?");
2 switch (arg) {
3   case '0':
4   case '1':
5     alert( 'One or zero' );
6     break;
7
8   case '2':
9     alert( 'Two' );
10    break;
11
12   case 3:
13     alert( 'Never executes!' );
14     break;
15   default:
16     alert( 'An unknown value' );
17 }
```

1. For `0`, `1`, the first `alert` runs.
2. For `2` the second `alert` runs.
3. But for `3`, the result of the `prompt` is a string `"3"`, which is not strictly equal `===` to the number `3`. So we've got a dead code in `case 3`! The `default` variant will execute.



Functions

- Quite often we need to perform a similar action in many places of script
- Functions are the main “building blocks” of the program
- They allow the code to be called many times without repetition
- We’ve already seen examples of built-in functions, like `alert(message)`, `prompt(message, default)` and `confirm(question)`



Function Declaration

- To create a function we can use a function declaration

```
1 function name(parameters) {  
2     ...body...  
3 }
```

```
1 function showMessage() {  
2     alert( 'Hello everyone!' );  
3 }  
4  
5 showMessage();  
6 showMessage();
```



Local variables

- A variable declared inside a function is only visible inside that function

```
1  <!DOCTYPE html>
2  <script>
3  'use strict';
4
5  function showMessage() {
6      let message = "Hello, I'm JavaScript!"; // local variable
7
8      alert( message );
9  }
10
11  showMessage(); // Hello, I'm JavaScript!
12
13  alert( message ); // <-- Error! The variable is local to the
    function
14  </script>
```



Outer variables

- A function can access and modify an outer variable as well

```
5  let userName = 'John';
6
7  function showMessage() {
8      userName = "Bob"; // (1) changed the outer variable
9
10     let message = 'Hello, ' + userName;
11     alert(message);
12 }
13
14 alert( userName ); // John before the function call
15
16 showMessage();
17
18 alert( userName ); // Bob, the value was modified by the
    function
19 </script>
```



Outer and local variables

- The outer variable is only used if there's no local one.
- If a same-named variable is declared inside the function then the outer one is ignored

```
5   let userName = 'John';
6
7   function showMessage() {
8       let userName = "Bob"; // declare a local variable
9
10      let message = 'Hello, ' + userName; // Bob
11      alert(message);
12  }
13
14  // the function will create and use its own userName
15  showMessage();
16
17  alert( userName ); // John, unchanged, the function did not
    access the outer variable
18  </script>
```



Global variables

- Variables declared outside of any function, such as the outer userName are called **global**
- Global variables are visible from any function (unless shadowed by locals)
- It's good practice to minimize the use of global variables
- Modern code has few or no globals
 - Sometimes though, globals can be useful to store project-level data



Parameters

- We can pass arbitrary data to functions using parameters (also called function arguments)

```
5  function showMessage(from, text) { //  
    arguments: from, text  
6  |  alert(from + ': ' + text);  
7  |  }  
8  
9  showMessage('Ann', 'Hello!'); // Ann: Hello!  
    (*)  
10 showMessage('Ann', "What's up?"); // Ann:  
    What's up? (**)  
11 </script>
```



Default values

- If a parameter is not provided, then its value becomes undefined
 - For instance, the function showMessage(from, text) can be called with a single argument

```
1  showMessage("Ann");
```

- Such a call would output “Ann: undefined”
 - There’s no text, so it’s assumed that text === undefined
- If we want to use a “default” text, we can specify it after =

```
5  function showMessage(from, text = "no text  
   given") {  
6  |    alert( from + ": " + text );  
7  | }  
8  
9  showMessage("Ann"); // Ann: no text given
```



Alternative default parameters

- To check for an omitted parameter, we can compare it with undefined

```
5  function showMessage(text) {  
6      if (text === undefined) {  
7          text = 'empty message';  
8      }  
9  
10     alert(text);  
11 }  
12  
13 showMessage(); // empty message
```

```
function showMessage(from, text)  
{  
    text = text || "no text";  
    alert(from + ": " + text);  
}
```



Using ?? to check undefined parameters

- Modern JavaScript engines support the nullish coalescing operator ??

```
5    // if there's no "count" parameter, show
    "unknown"
6    function showCount(count) {
7    |    alert(count ?? "unknown");
8    |    }
9
10   showCount(0); // 0
11   showCount(null); // unknown
12   showCount(); // unknown
```



Returning a value

- A function can return a value back into the calling code as the result
- A function with an empty return or without it returns undefined

```
1 function doNothing() { /* empty */ }  
2  
3 alert( doNothing() === undefined ); // true
```

An empty `return` is also the same as `return undefined` :

```
1 function doNothing() {  
2     return;  
3 }  
4  
5 alert( doNothing() === undefined ); // true
```



Never add a newline between return and the value

- For a long expression in return, we should start it at the same line as return or at least put the opening parentheses there as follows

```
1  return (  
2      some + long + expression  
3      + or +  
4      whatever * f(a) + f(b)  
5  )
```



Naming a function

- Functions are actions. So their name is usually a verb
- It should be brief, as accurate as possible and describe what the function does, so that someone reading the code gets an indication of what the function does

```
1 showMessage(..)      // shows a message
2 getAge(..)           // returns the age (gets it somehow)
3 calcSum(..)          // calculates a sum and returns the result
4 createForm(..)       // creates a form (and usually returns it)
5 checkPermission(..)  // checks a permission, returns true/false
```



One function – one action

- A function should do exactly what is suggested by its name, no more
- Two independent actions usually deserve two functions, even if they are usually called together (in that case we can make a 3rd function that calls those two).
- A few examples of breaking this rule:
 - `getAge` – would be bad if it shows an `alert` with the age (should only get).
 - `createForm` – would be bad if it modifies the document, adding a form to it (should only create it and return).
 - `checkPermission` – would be bad if it displays the `access granted/denied` message (should only perform the check and return the result).

Summary

- A function declaration looks like this

```
1  function name(parameters, delimited, by, comma) {  
2      /* code */  
3  }
```

- Values passed to a function as parameters are copied to its local variables
- A function may access outer variables
- A function can return a value. If it doesn't, then its result is undefined



References

- <https://javascript.info/>

